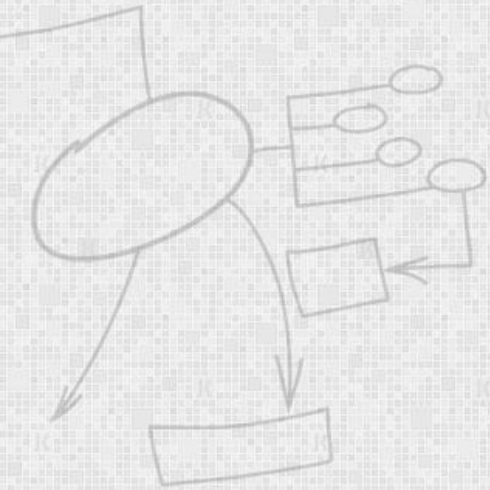
EC-360
SOFTWARE ENGINEERING

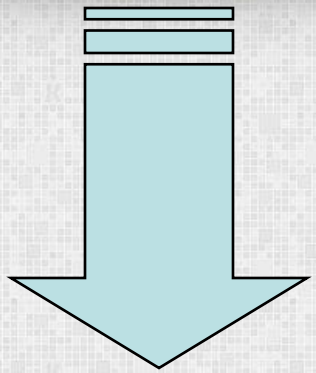


Lecture 11

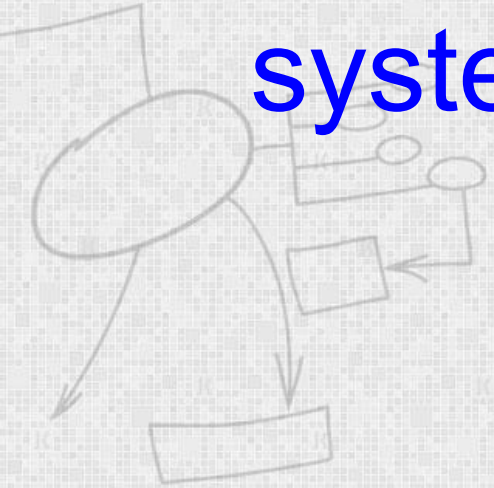
Software Testing

By
Dr Wasi Haider Butt

Verification and Validation



Assuring that a software system meets a user's needs



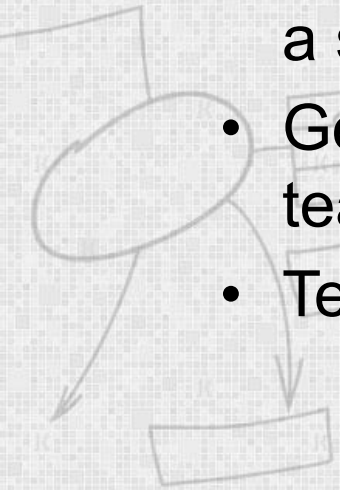
Verification vs Validation

- **Verification**:
  "Are we building the product right"
- The software should conform to its specification

- **Validation**:
  "Are we building the right product"
- The software should do what the user requires correctly.

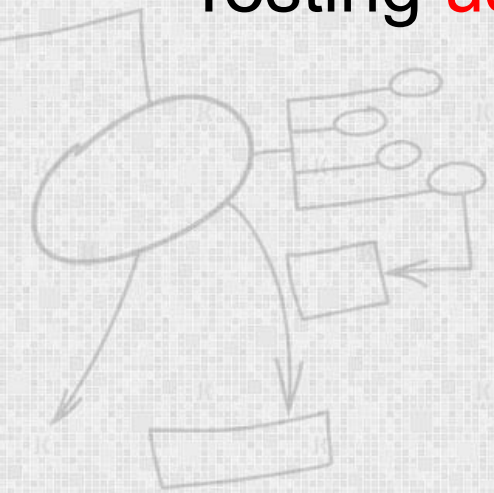
The testing process

- Component testing
  - Testing of individual program components
  - Usually the responsibility of the component developer (except sometimes for critical systems)
- Integration testing
  - Testing of groups of components integrated to create a system or sub-system
  - Generally the responsibility of an independent testing team
  - Tests are based on a system specification



Testing priorities

- Only exhaustive testing can show a program is free from defects. However, exhaustive testing is impossible
- Testing old capabilities is more important than testing new capabilities
- Testing usual/unusual situations



Test data, cases and suites

- *Test data* Inputs which have been devised to test the system
- *Test cases* Inputs to test the system and the predicted outputs from these inputs if the system operates according to its specification
- Collection of different test cases in Test Suite

P = Plain Text | C = Cipher Text | K = Key

Test Case ID	Test Case Objective	Pre requisite	Steps	Input Data	Expected Output	Actual Output	Status
TC_01	Test Caesar Cipher Algorithm (For Encryption)	Textfield should be enabled	1. Select Encrypt Button 2. Enter Plain Text 3. Enter Numeric Key 4. Submit	P: hello K: 3	khoor	khoor	PASS
TC_02	Test Caesar Cipher Algorithm (For Decryption)	Textfield should be enabled	1. Select Decrypt Button 2. Enter Cipher Text 3. Enter Numeric Key 4. Submit	C: khoor K: 3	hello	hello	PASS
TC_03	Test Vignere Cipher	Text Fields should be enabled	1. Enter Plain Text 2. Enter String Key 3. Submit	P: hello K: abcds	hfnog	fhgon	FAIL
TC_04	Test MD5	Text Fields should be enabled	1. Enter Plain Text 2. Submit	T: hello	5d41402a bc4b2a76 b9719d91 1017c592	5d41402a bc4b2a76 b9719d91 1017c592	PASS
TC_05	Test Columner Cipher	Text Fields should be enabled	1. Enter Plain Text 2. Enter Numeric Key 3. Submit	P: hello K: 3	hleol	hleol	PASS

Design of test cases

- Exhaustive testing of almost any system is impractical since the domain of input data values to most practical software systems is either extremely large or infinite.
- Therefore, we must design an optimal test suite that is of reasonable size and can uncover as many errors existing in the system as possible.

Design of test cases

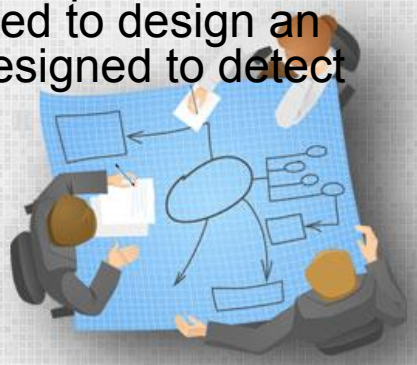
- Consider the following example code segment which finds the greater of two integer values x and y. This code segment has a simple programming error.

  - **If (x>y) max = x;**
  - **else max = x;**

- For the above code segment, the test suite, {(x=3,y=2);(x=2,y=3)} can detect the error.

- whereas a larger test suite {(x=3,y=2);(x=4,y=3);(x=5,y=1)} does not detect the error.

- This implies that the test suite should be carefully designed than picked randomly. Therefore, systematic approaches should be followed to design an optimal test suite. In an optimal test suite, each test case is designed to detect different errors.



Functional testing vs Structural testing

- In the black-box testing approach, test cases are designed using only the functional specification of the software, i.e. without any knowledge of the internal structure of the software. For this reason, black-box testing is known as functional testing.
- On the other hand, in the white-box testing approach, designing test cases requires thorough knowledge about the internal structure of software, and therefore the white-box testing is called structural testing.

Black box testing

- In the black-box testing, test cases are designed from an examination of the input/output values only and no knowledge of design, or code is required.

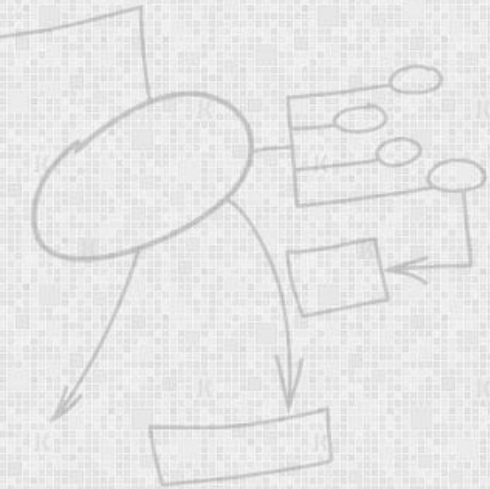
- Test cases are designed in two steps
  1. Equivalence class portioning
  2. Boundary value analysis

Equivalence class portioning

- In this step, the domain of input values to a program is partitioned into a set of equivalence classes.
- The main idea behind defining the equivalence classes is that testing the code with any one value belonging to an equivalence class is as good as testing the software with any other value belonging to that equivalence class.
- Equivalence classes for a software can be designed by examining the input data and output data.

Equivalence Class Partitioning

- If the input data values to a system can be specified by a range of values, then one valid and two invalid equivalence classes are recommended

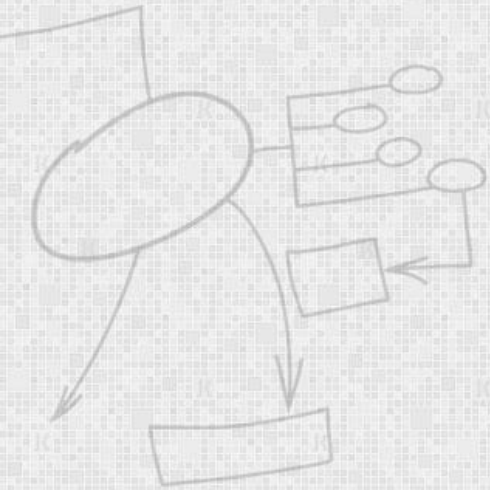


Equivalence Class Partitioning

- For a software that computes the square root of an input integer which can assume values in the range of 0 to 5000, there are three equivalence classes:
- The set of negative integers, the set of integers in the range of 0 and 5000, and the integers larger than 5000.
- Therefore, the test cases must include representatives for each of the three equivalence classes and a possible test set can be: {-5,500,6000}.

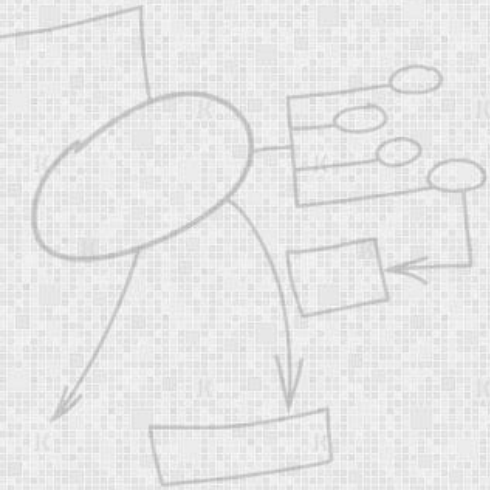
Boundary value analysis

- Boundary value analysis leads to selection of test cases at the boundaries of the different equivalence classes.



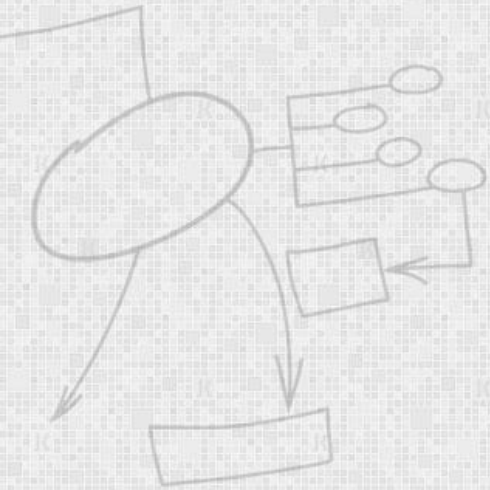
Boundary Value Analysis

- Example: For a function that computes the square root of integer values in the range of 0 and 5000.
- The test cases must include the following values: {-1, 0,5000,5001}.



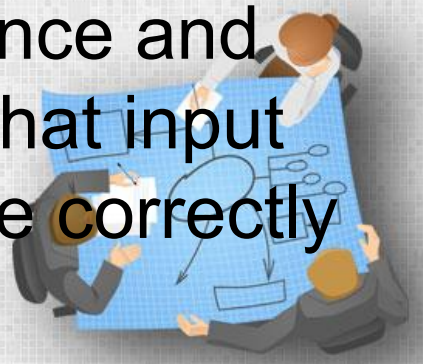
White-Box Testing

- Three Techniques
  - Statement coverage
  - Branch coverage
  - Path coverage



Statement coverage

- The statement coverage strategy aims to design test cases so that every statement in a program is executed at least once.
- The principal idea governing the statement coverage strategy is that unless a statement is executed, it is very hard to determine if an error exists in that statement.
- However, executing some statement once and observing that it behaves properly for that input value is no guarantee that it will behave correctly for all input values.



Statement coverage

- Consider the Euclid's GCD computation algorithm:

```
int compute_gcd(x, y)
int x, y;
{
1  while (x! = y){
2  if (x>y) then
3  x= x – y;
4  else y= y – x;
5  }
6  return x;
}
```

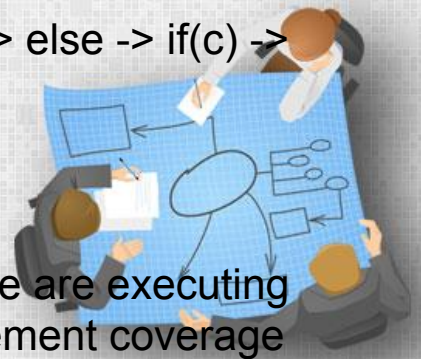
- By choosing the test set (x=4, y=3) or (x=3, y=4)}, we can exercise the program such that all statements are executed at least once.

Branch coverage

- In the branch coverage-based testing strategy, test cases are designed to make each branch condition to assume true and false values in turn.
- It is obvious that branch testing guarantees statement coverage and thus is a stronger testing strategy compared to the statement coverage-based testing.
- For Euclid's GCD computation algorithm , the test cases for branch coverage can be {(x=3, y=3) (x=3, y=2) (x=4, y=3) (x=3, y=4)}.

Branch coverage

- If (a || b)
  - { test1 = true; }
- else
  - {
    - if(c)
      - { test2 = true; }
  - }
- **statement coverage** have to test each statement at least once, so we need just two tests:
  - a=true b=false - that gives us path if(a||b) true -> test1 = true
  - a=false, b=false and c=true - that gives us path: if(a||b) false -> else -> if(c) -> test2=true.
- This way we executed each and every statement.
- **Branch coverage** needs one more test:
  - a=false, b=false, c=false - that leads us to that second if but we are executing false branch from that statement, that wasn't executed in statement coverage

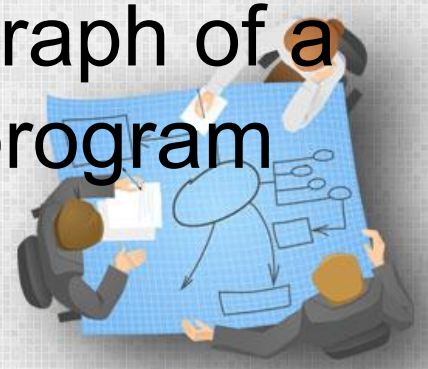


Path coverage

- The path coverage-based testing strategy requires us to design test cases such that all linearly independent paths in the program are executed at least once.
- A linearly independent path can be defined in terms of the control flow graph (CFG) of a program.

Control Flow Graph (CFG)

- A control flow graph describes the sequence in which the different instructions of a program get executed.
- In other words, a control flow graph describes how the control flows through the program.
- In order to draw the control flow graph of a program, all the statements of a program must be numbered first.



Control Flow Graph (CFG)

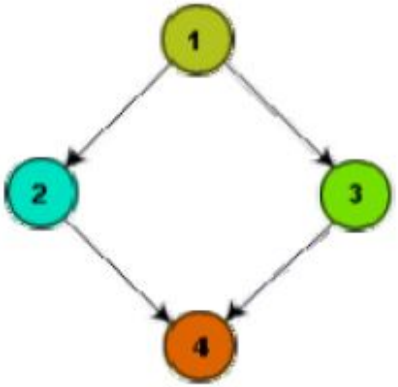
Sequence:

```
1. a=5;
2. b=a*2-1;
```

(a)

Selection:

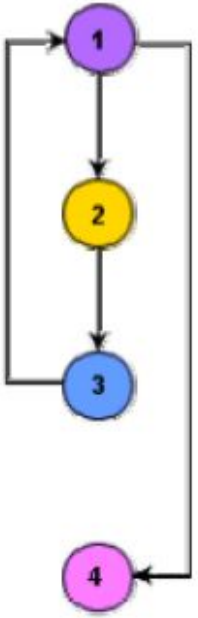
```
1. if(a>b)
2. c=3;
3. else c=5;
4. c=c*c;
```



(b)

Iteration:

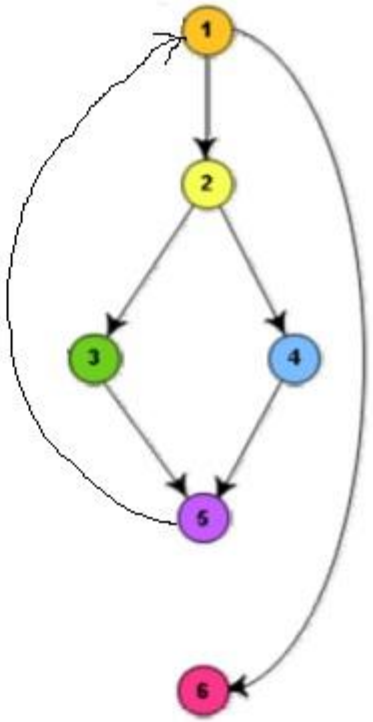
```
1. while(a>b){
2. b=b-1;
3. b=b*a;}
4. c=a+b;
```



(c)

Control Flow Graph (CFG)

```
int compute_gcd(int x, int y){
1 while(x! =y){
2       if(x>y) then
3               x=x-y;
4       else  y=y-x;
5 }
6 return x;
}
```



Control Flow Graph (CFG)

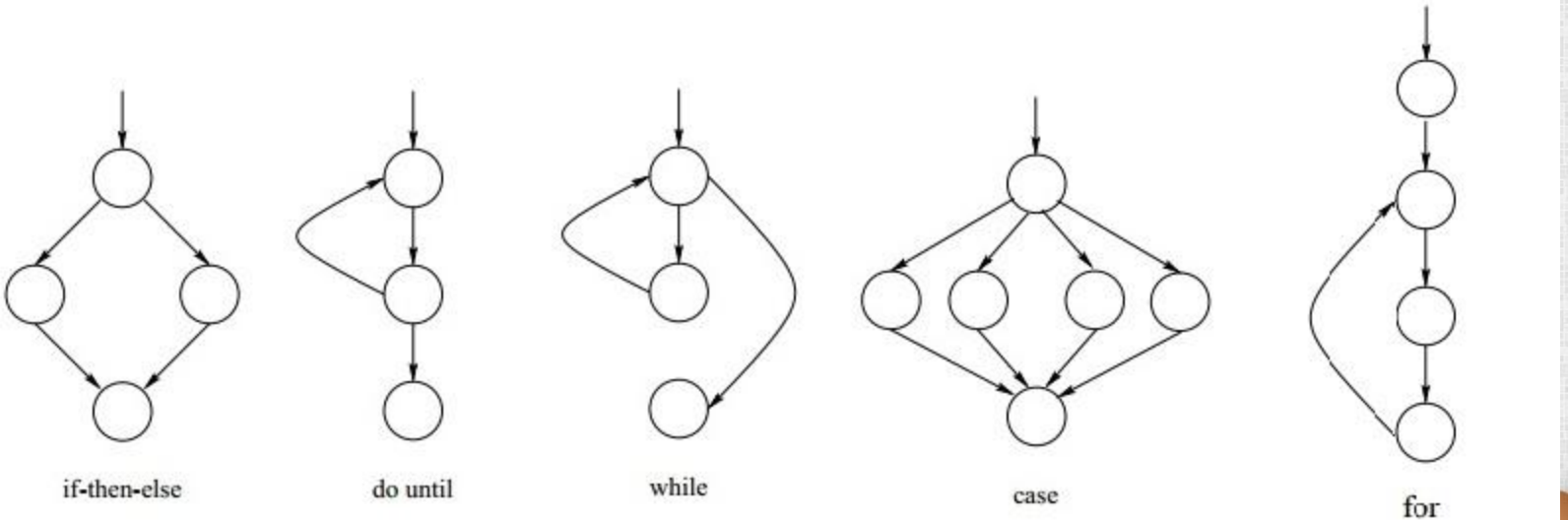
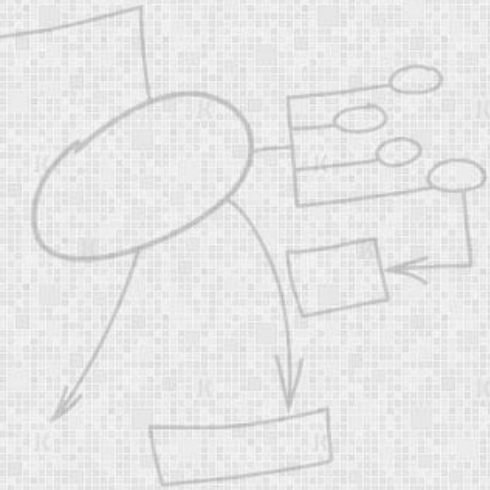


Figure 1: Flow graph representation.

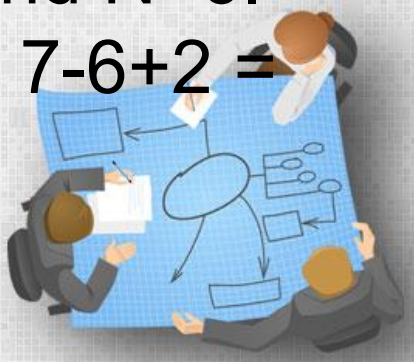
Cyclomatic complexity

- McCabe's cyclomatic complexity metric provides a practical way of determining the maximum number of linearly independent paths in a program.
- There are three different ways to compute the cyclomatic complexity. The answers computed by the three methods are guaranteed to agree.



Cyclomatic complexity

- **Method 1:**
  - Given a control flow graph G of a program, the cyclomatic complexity V(G) can be computed as:
  - V(G) = E – N + 2
  - where N is the number of nodes of the control flow graph and E is the number of edges in the control flow graph.
- For the CFG of example shown, E=7 and N=6. Therefore, the cyclomatic complexity = 7-6+2 = 3.

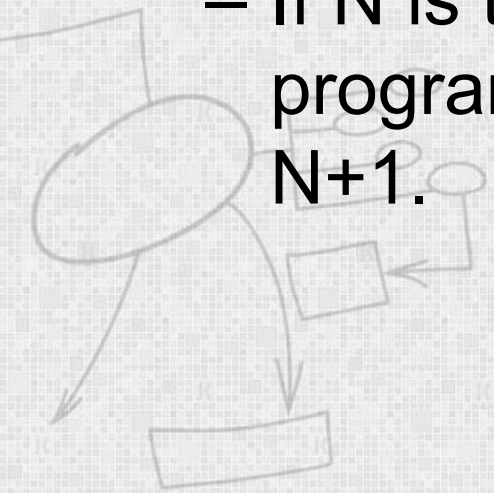


Cyclomatic complexity

- **Method 2:**
  - An alternative way of computing the cyclomatic complexity of a program from an inspection of its control flow graph is as follows:
  - **V(G) = Total number of bounded areas + 1**
- For the CFG example, from a visual examination of the CFG the number of bounded areas is 2.
- Therefore the cyclomatic complexity, computing with this method is also 2+1 = 3.

Cyclomatic complexity

- **Method 3:**
  - The cyclomatic complexity of a program can also be easily computed by computing the number of decision statements of the program.
  - If N is the number of decision statement of a program, then the McCabe's metric is equal to N+1.



Cyclomatic complexity And Path Coverage

- Number of test cases should be equal to cyclometic complexity
- Test case should be designed in such a way that all linearly independent paths are covered
- Cyclometic complexity of a good code is recommended to be less than 10

NFRs testing

- NFRs testing is carried out to check whether the system meets the non-functional requirements identified in the SRS document.
- There are several types of NFRs testing.
- Mostly NFRs tests can be considered as black-box tests.

NFRs testing

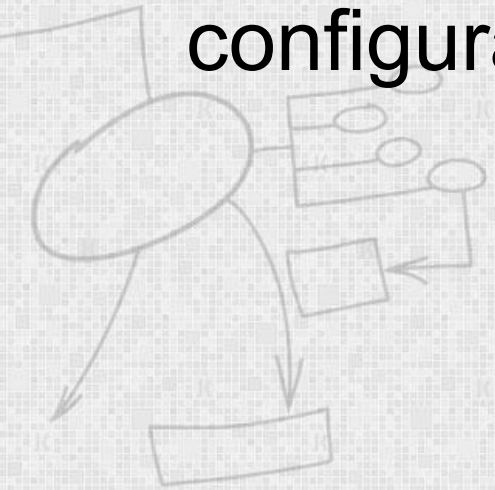
- Stress testing
- Configuration testing
- Compatibility testing
- Regression testing
- Documentation testing
- Usability testing

Stress Testing

- Stress testing is also known as endurance testing.
- Stress testing evaluates system performance when it is stressed.
- Stress tests are black box tests which are designed to impose a range of abnormal and even illegal input conditions so as to stress the capabilities of the software.
- Stress testing is especially important for systems that usually operate below the maximum capacity but are severely stressed at some peak demand hours.

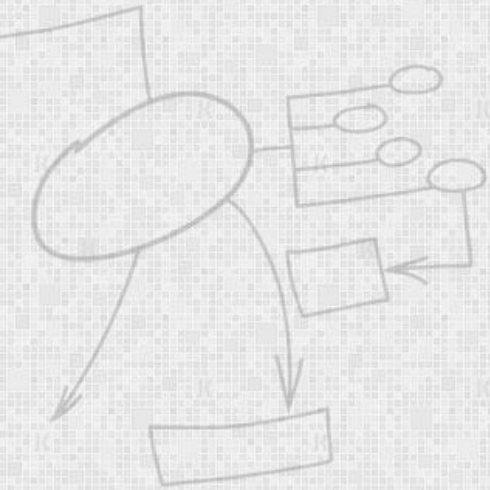
Configuration Testing

- This is used to analyse system behaviour in various hardware and software configurations specified in the requirements.
- Sometimes systems are built in variable configurations for different users.



Compatibility Testing

- This type of testing is required when the system interfaces with other types of systems.
- Compatibility aims to check whether the interface functions perform as required.



Regression Testing

- This type of testing is required when the system being tested is an upgradation of an already existing system to fix some bugs or enhance functionality, performance, etc.
- Regression testing is the practice of running an old test suite after each change to the system or after each bug fix to ensure that no new bug has been introduced due to the change or the bug fix.

Documentation Testing

- It is checked that the required user manual, maintenance manuals, and technical manuals exist and are consistent.
- If the requirements specify the types of audience for which a specific manual should be designed, then the manual is checked for compliance.

Usability Testing

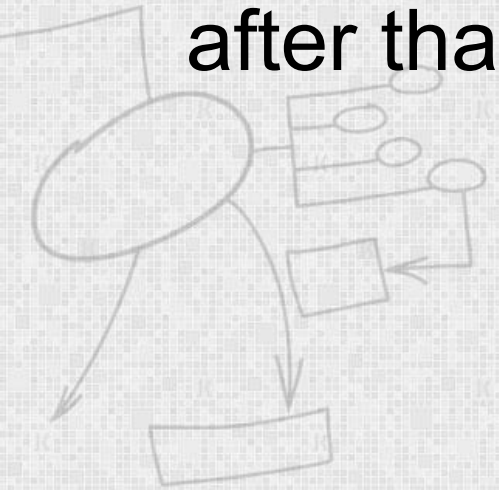
- Usability testing concerns checking the user interface to see if it meets all user requirements concerning the user interface.
- During usability testing, the display screens, report formats, and other aspects relating to the user interface requirements are tested.

Test Automation

- Test automation is the use of software
  - *To set test preconditions.*
  - *To control the execution of tests.*
  - To *compare the actual outcomes to predicted outcomes.*
  - *To report the Execution Status.*
- Commonly, test automation involves automating a manual process already in place that uses a formalized testing process.

Test Automation

- If a manual test costs $X to run the first time, it will cost $X to run every time thereafter.
- An automated test can cost 3 to 30 times $X the first time, but will cost about $0 after that.



Test Automation Tools

- Quick Test Professional By HP
- Rational Functional Tester By Rational (IBM Company)
- Silk Test By Borland
- Test Complete By Automated QA
- QA Run (Compuware)
- Watir (Open Source)
- Selenium (Open Source)
- Sahi (Open Source)